# HeatingSystemController
# Fejleszti dokumentáció

AlfaOmega projekt

November 6, 2026

## Contents

# 1 Dokumentum célja

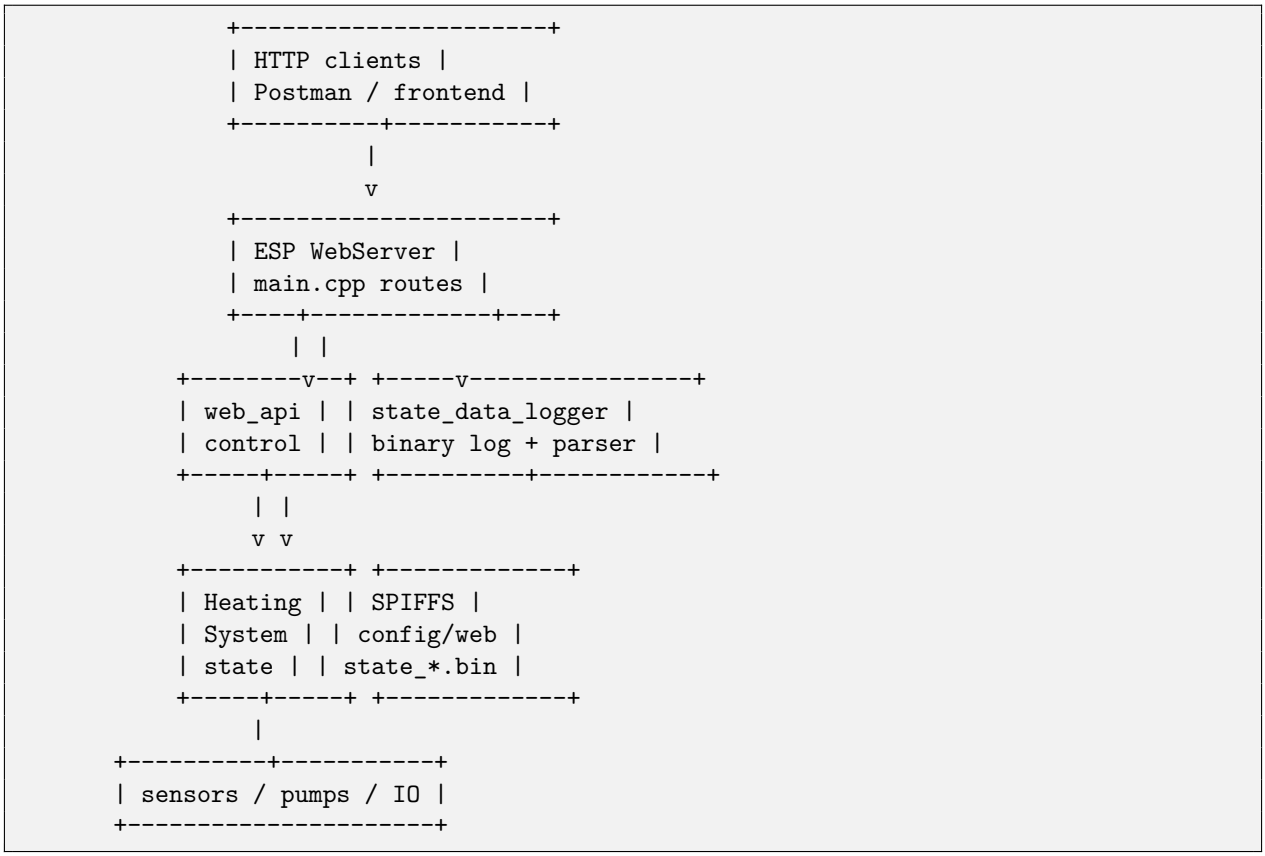
Ez a dokumentum a HeatingSystemController firmware továbbfejlesztéséhez szükséges technikai információkat foglalja össze. Ismerteti a futási architektúrát, a ftési rendszer modelljét, a HTTP API-kat, a konfigurációs formátumot, a SPIFFS fájlrendszert, a bináris állapotnapló formátumát, a frontend adat-visszaépítését, a build környezeteket és az integrációs teszteket.

A projekt f forrásfájljai:

- `src/main.cpp`: inicializálás, f ciklus, HTTP és WebSocket életciklus.
- `src/web_api.cpp`: állapot-, konfigurációs-, hmérséklet- és vezérl API-k.
- `src/config.h`: konfiguráció betöltése, validálása és feltöltése.
- `src/HeatingSystemElements/`: kazán, puffer, radiátor, szenzor és vezérl objektumok.
- `src/state_data_logger.cpp`: bináris napi állapotnapló és JSON parser API.
- `config/config.json`: fejleszti konfigurációs példa.
- `data/config.json`: a firmware fájlrendszerébe feltöltend konfiguráció.

# 2 Rendszerarchitektúra

Az alkalmazás Arduino frameworkre épül, PlatformIO-val fordítható ESP8266 és ESP32 célokra. A firmware a SPIFFS fájlrendszert használja webes fájlok, konfiguráció és runtime naplófájlok tárolására.



### 2.1 Inicializálási sorrend

A `setup()` f lépései:

1. soros port és hardver inicializálása;
2. SPIFFS és `config.json` betöltése;
3. `HeatingSystem` objektum létrehozása;
4. WiFi és NTP kliens inicializálása;
5. HTTP route-ok regisztrálása;
6. szenzorok és ftési elemek inicializálása;
7. `StateDataLogger::begin()` meghívása;
8. WebSocket és idzít indítása.

Az `loop()` minden ciklusban kezeli a hálózatot, frissíti az NTP idt és a Dallas szenzorokat, feldolgozza a ftési rendszer jelzéseit, majd meghívja a `stateDataLogger.capture()` függvényt.

## 3 Ftési rendszer modellje

A `HeatingSystem` a konfigurációból létrehozza a ftési elemeket és a közös `mergedList` listában tartja ket. A f elemtípusok:

- **Kazan**: kazán állapotgép, visszatér-védelem és túlmelegedés-védelem.
- **Puffer**: töltés, kisütés, készenlét és telítettségi állapotok.
- **Radiator**: fogyasztói oldal állapotgépe.
- **HeatingController**: a teljes rendszer módja és higénye.

A szenzorok a `Sensor::SensorsValue` mapben jelennek meg az egyedi DS18B20 azonosítójukkal. A kimenetek az `IOMap` mapben vannak, ahol a kulcs az IO szám, az érték tipikusan 0–100 közötti PWM vagy ON/OFF érték.

A modulállapotok a `HeatingElement::ElementsStateMap` mapben vannak. Gyakori mezk:

- `state`: állapotnév, például `CHARGE`, `DISCHARGE`, `HEATING`;
- `HT`: hátadási érték, floatként naplózva;
- `NH`: higény;
- `iA`: aktív állapot;
- `iOHPA`, `iRPA`: kazánvédelmi állapotok.

## 4 Konfiguráció

A konfiguráció JSON fájlban található a SPIFFS gyökerében `/config.json` néven. A projektben található példa a `config/config.json` és `data/config.json` fájlban van.

### 4.1 Globális mezk

| Mez | Jelentés |
|---|---|
| `wifiSSID` | STA módú WiFi hálózat neve. |
| `wifiPassword` | STA módú WiFi jelszó; publikus dokumentációban ne szerepeljen valódi érték. |
| `AP_SSID` | Access Point mód hálózatneve. |
| `AP_Password` | Access Point jelszava. |
| `WifiMode` | `STA`, `AP` vagy `STA+AP`. |
| `hostname` | DHCP/mDNS név. |
| `comType` | `API` vagy `WS`; WebSocket esetén a frontend állapotbroadcastot is kaphat. |

### 4.2 Ftési elemek

A `HeatingSystem` objektumai a `KAZAN`, `RADIATOR` és `PUFER` tömbökben vannak. Egy szenzor mezi:

```
{
  "model": "DS12B20",
  "position": "body",
  "id": "2814576B00000000059",
  "offset": -1.0,
  "level": 80
}
```

A `position` értéke `tour`, `retour` vagy `body`. Több body szenzor esetén a `level` súlyozza az átlagot.

A pumpa fontos mezi a `name`, `workingMode`, `IOnumber`, `minControlSig`, `maxControlSig` és opcionálisan a `dischargepump`. PID használatakor a `pid.enabled`, `p`, `i`, `d`, `tempSensorId` és `setPoint` mezk kötelezek.

## 5 HTTP API

Az API alapértelmezett portja 80. A route-ok regisztrációja a `src/main.cpp` `registerHttpRoutes()` függvényében történik.

### 5.1 API-kérés limit

Az API limiter a `API_MIN_REQUEST_INTERVAL_MS` build flaggel konfigurálható. Alapértéke 0, ezért jelenleg ki van kapcsolva.

```
-DAPI_MIN_REQUEST_INTERVAL_MS=250
```

Bekapcsoláskor a túl gyors kérés HTTP 429 választ és `Retry-After` fejlécet ad.

### 5.2 Állapot lekérése

```
GET /state
```

A válasz egy JSON objektum. Az IO-k és szenzorok lapos kulcsként, a modulállapotok modulnév alatt jelennek meg.

```
{
  "1": 0,
  "3": 100,
```

```
  "28000B96A00000000029": 55.4,
  "Puffer950l": {
    "state": "CHARGE",
    "HT": "12.3"
  }
}
```

### 5.3 Hmérsékletértékek beállítása

```
POST /setTemp
Content-Type: application/json
```

Példa:

```
{
  "28000B96A00000000029": 55.0,
  "2814576B00000000059": 48.5
}
```

Sikeres válasz: `"status":"success"`. Ezt a tesztkörnyezet használja az érzékelértékek szimulálására.

### 5.4 Ftési igény

```
GET /heatingRequest?name=PadloKozos&setTemp=true
GET /heatingRequest?name=PadloKozos&setTemp=false
```

A `name` konfigurált ftési elemnév, a `setTemp` ebben az API-ban logikai ftési igényt jelent.

### 5.5 Manuális pumpa- és IO-vezérlés

```
POST /manualControlPumps
Content-Type: application/json
```

Aktuális formátum:

```
{
  "3": {"manual": true, "value": 50},
  "4": {"manual": false}
}
```

Kompatibilitási formátum is támogatott:

```
{"3": 50, "4": 0}
```

A PWM értéknek 0 és 100 közé kell esnie. A manuális mód felülírja az automatikus vezérlést az adott IO-n.

### 5.6 Eszközinformáció

```
GET /deviceInfo
```

A válasz tartalmazza a firmware uptime értékét, WiFi adatokat, chip- és flash-információkat, SPIFFS teljes/használt/szabad méretét, heap adatokat és reset információt.

### 5.7 Konfiguráció lekérése és feltöltése

```
GET /configuration
POST /configuration
```

A GET a jelenlegi `/config.json` fájlt adja vissza. A POST multipart file upload, a fájl neve legyen `config.json`. A firmware validálja a konfigurációt, biztonsági mentést készít, majd a sikeres feltöltés után újraindítás szükséges.

### 5.8 Újraindítás

```
GET /reboot?user=admin&password=password
```

Ez a jelenlegi implementációban rögzített hitelesítést használ. Éles rendszerben ezt módosítani kell.

## 6 Bináris állapotnapló

A naplózás a `StateDataLogger` osztályban van. A fájlnév:

```
/state_YYYY-MM-DD.bin
```

A napló az NTP kliens által szolgáltatott dátum alapján napi fájlba kerül. A naplózás legalább másodpercenként ellenriz, de csak változás esetén ír új mintát. A hmérsékleti küszöb 0,1 °C, build flaggel módosítható.

### 6.1 Fájlfejléc

| Offset | Méret | Tartalom |
|---|---|---|
| 0 | 4 bájt | `HLOG` magic |
| 4 | 1 bájt | formátumverzió, jelenleg 1 |
| 5 | 3 bájt | fenntartott |

### 6.2 Rekordfejléc

Minden rekordhoz 5 bájtos fejléc tartozik:

| Méret | Típus | Jelentés |
|---|---|---|
| 1 bájt | `uint8_t` | rekordtípus: 1 séma, 2 minta |
| 2 bájt | `uint16_t` | payload hossza, little-endian |
| 2 bájt | `uint16_t` | CRC16 a payloadra |

### 6.3 Sémarekord

A séma azonosítókat rendel az emberileg értelmezhet kulcsokhoz. Egy mez leírása 2 bájt ID-t, 1 bájt típust, 1 bájt névhosszt és a meznév bájtjait tartalmazza. Példák:

```
io/3
sensor/2814576B00000000059
state/Puffer950l/state
state/Puffer950l/HT
```

Sémarekord akkor kerül a fájlba, amikor a logger elször indul, új nap kezddik, vagy az aktuális mezkészlet/típus megváltozik. A konfigurációból késbb megjelen új szenzor új sémát eredményezhet.

### 6.4 Mintarekord

Egy mintarekord egy idpontot és az adott idpontban megváltozott mezket tartalmazza. Az els idbélyeg teljes 32 bites Unix id, a továbbiak Delta-of-Delta ZigZag-varint formában vannak. Egy mez 2 bájtos azonosítóból, 1 bájtos típusból és 1 vagy 4 bájtos értékbl áll.

- `uint8`: IO/PWM vagy enum/állapotérték;
- `float32`: hmérséklet és `HT` érték;
- egy mintában több mez osztozik az idbélyegen és a CRC-n.

### 6.5 JSON visszaolvasás

```
GET /state-history/files
GET /state-history?date=YYYY-MM-DD
DELETE /state-history?date=YYYY-MM-DD
```

A lekérdezés válasza változáslistát ad:

```
[
  {
    "timestamp": 1793913984,
    "changes": {
      "io/1": 0,
      "sensor/2814576B00000000059": 48.5,
      "state/Puffer950l/state": "CHARGE"
    }
  }
]
```

A frontendnek nem szabad a `changes` objektumot teljes állapotként kezelnie. A rekordokat idrendben kell feldolgozni, és minden változást az elz teljes állapotba kell merge-elni.

## 7 SPIFFS és tárhely

ESP8266 esetén a `esp12e` környezet alapértelmezett flash-elrendezést használ. ESP32 esetén a `partitions_4mb_web.csv` SPIFFS partíciója `0x230000` bájt, azaz 2 293 760 bájt. A ténylegesen szabad helyet a `GET /deviceInfo` válasz `spiffs.total`, `spiffs.used` és `spiffs.free` mezi adják.

A naplófájlok automatikus törlése jelenleg nincs bekapcsolva. Több nap után a frontend vagy egy karbantartási feladat használhatja a DELETE végpontot.

## 8 Build és telepítés

Gyakori környezetek a `platformio.ini` fájlban:

| Környezet | Cél |
|---|---|
| `esp12e` | ESP8266 normál firmware |
| `esp12e_testOverAPI` | ESP8266 API integrációs tesztekhez |
| `esp12e_testRandomData` | ESP8266 szintetikus/random adatokhoz |
| `esp32dev` | ESP32 normál firmware |
| `esp32dev_testOverAPI` | ESP32 API integrációs tesztekhez |
| `linux_test` | natív unit teszt környezet |

```
pio run -e esp12e_testOverAPI
pio run -e esp12e_testOverAPI -t upload
pio run -e esp12e_testOverAPI -t uploadfs
```

A naplózási build flag-ek:

```
-DSTATE_LOG_TEMP_THRESHOLD_C=0.1f
-DSTATE_LOG_MIN_CAPTURE_INTERVAL_MS=1000UL
-DAPI_MIN_REQUEST_INTERVAL_MS=0UL
```

# 9 Tesztelés

A Python integrációs tesztek a `tests/` könyvtárban vannak. A firmware-nek hálózaton elérhetnek kell lennie.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-test.txt
HEATING_CONTROLLER_URL=http://192.168.0.106 \\
  python -m pytest -m integration -vv -s
```

A tesztek alapértelmezett beállási várakozása 3 másodperc. A részletes API riportokhoz használható a `-api-trace`, `-api-report` és `-api-html-report` kapcsoló.

# 10 Továbbfejlesztési javaslatok

1. Induláskor a meglév fájl utolsó sémájának beolvasása, hogy újraindításkor ne kerüljön be feleslegesen azonos sémarekord.
2. A fájllista végpont kiegészítése teljes, használt és szabad SPIFFS mérettel.
3. Napló-retenció vagy maximális naplófájl-szám bevezetése.
4. A `/reboot` hitelesítésének konfigurálhatóvá tétele.
5. A state enumok központi, verziózott dictionary-je az ismeretlen állapotok hash-elése helyett.
6. A bináris formátumhoz host oldali dekóder és automatikus formátumteszt készítése.